
Towards Perfect Code Generation: Iterative API-based Feedback Loops

Harman Preet Singh

852863316

Abstract

The abstract paragraph should be indented 1/2 inch (3 picas) on both the left- and right-hand margins. Use 10 point type, with a vertical spacing (leading) of 11 points. The word **Abstract** must be centered, bold, and in point size 12. Two line spaces precede the abstract. The abstract must be limited to one paragraph.

Keywords: keyword 1 · keyword 2 · keyword 3

1 Introduction (1 – 1.5 pages)

Open with the undeniable strength of LLMs at code generation — they produce syntactically valid, compilable, visually correct HTML. Then immediately introduce the tension: syntactic correctness is not the same as semantic correctness. The code works, but that doesn't mean it's right.

Explain the specific bad habit: overuse of `<div>` where semantically meaningful tags (`<section>`, `<article>`, `<nav>`, `<header>`, etc.) should be used. Keep it concrete — give a brief illustrative example of what "divitis" looks like and why a developer might not even notice it. Mention that LLMs are trained on real-world code, which already contains this pattern, making them likely to reproduce it.

Explain the consequences. Two angles: (1) maintainability — tags make codebases easier to read, maintain and extend; (2) accessibility — screen readers rely on semantic structure to navigate pages for users with visual impairments. This is where you make the societal relevance concrete. Keep it brief but pointed.

Transition to the problem space. Even if you instruct an LLM via a system prompt to use semantic tags, compliance is not guaranteed — especially for smaller or local models. There is currently no widely adopted automated mechanism that catches and corrects this class of semantic errors in LLM-generated HTML. This sets up your contribution without going into related work yet.

Introduce the guardrail concept in one short paragraph: an iterative validation-and-regeneration feedback loop. The W3C validator flags issues; the LLM is reprompted with its own errors; the loop runs until the output is clear or a maximum iteration count is reached. Do not go into implementation details here — just enough to orient the reader.

State RQ1 and RQ2 explicitly. Then close with a one or two sentence roadmap: "The remainder of this report is structured as follows: Section 2 reviews related work, Section 3 presents the methodology..." and so on.

2 Related Work (2 – 3 pages)

Brief orienting paragraph. Signal that the related work spans three interconnected areas: (1) LLM code generation quality, (2) iterative self-correction and feedback loops, and (3) HTML semantic quality and validation. Tell the reader you'll cover each in turn. This prevents the section from reading as a disconnected list of papers.

Discuss what the literature says about LLMs generating code — their strengths, but also the documented tendency to reproduce patterns from training data including bad practices. This is where you cite the Souly et al. [1] poisoning paper from your proposal. Expand beyond that one source: are there other papers documenting LLM code quality issues or bias towards common but suboptimal patterns? End with a critical note: most

evaluation work focuses on functional correctness (does it run?) rather than semantic or stylistic correctness — this is a gap your work addresses.

This is the densest part of the related work. Cover the landscape of your approaches where LLMs are given feedback and asked to revise their output. The paper your supervisor linked ([2]) is central here — discuss how it uses multiple feedback models rather than a single validator. Contrast it with your approach: you use an external, deterministic validator rather than another LLM as the feedback source, which avoids introducing a second model's biases. Identify what the literature agrees on (iterative feedback generally helps) and where there are gaps or contradictions (optimal iteration count, diminishing returns, cost trade-offs are rarely studied together).

Review any existing literature comparing model sizes on code tasks. What do prior studies say about whether smaller models can close the gap with large ones given more attempts or better prompting? If the literature is thin here (it likely is for this specific use case), say so explicitly — this positions RQ2 as a genuine open question rather than a settled one.

Discuss what is known about HTML semantic quality as a measurable property. Cover the W3C Markup Validation Service — what it does and does not detect. Be critical here: the validator is designed for syntactic and structural conformance, not semantic intent. Acknowledge this gap honestly. If there is prior work using automated validation as a proxy for code quality, discuss it and its limitations.

Close the section by pulling the threads together. No prior work combines an external deterministic validator, an iterative reprompting loop, a blind ablation control, and a cost-efficiency comparison across model sizes in the context of HTML semantic quality. This is your contribution's position in the literature. Keep this focused — one tight paragraph, not a repeat of the introduction.

3 Research Questions & Hypotheses (0.5 pages)

State RQ1 formally and precisely. Then in one or two sentences, give a brief rationale for why this question is answerable with your design — the before/after error counts and the feedback vs. blind ablation directly operationalise it.

State RQ2 formally. Clarify the operationalisation: "comparable quality" is defined as equivalent error reduction rates, and "cost" is measured via mean generation time per prompt per model. One sentence noting that cloud models are explicitly out of scope for this study, and why (all models run locally through Ollama).

Add brief hypotheses grounded in your initial testing. For example: reasoning-capable models are expected to converge faster regardless of parameter count; the feedback condition is expected to outperform the blind condition across all models; harder prompts are expected to require more iterations. These give the Results section something concrete to confirm or challenge, with the rubric rewards under "in-depth analysis".

4 Methodology (4 – 5 pages)

4.1 System Architecture

Describe the three components (LLM, validator, reprompting pipeline) and how they interact. Include your flowchart (Figure A1). Explain design choices, e.g. why Docker-hosted W3C rather than the public API.

4.2 Prompt Dataset

Describe how you constructed your prompt set. How many prompts? How did you decide which ones to include? Did you deliberately design some to provoke semantic tag errors (e.g. tasks that structurally require `<nav>`, `<article>`, etc.)? This directly addresses the supervisor's feedback about the random prompt selection being unclear.

Change: Remove the mention of random sampling — the dataset is now a fixed, ordered set of 51 prompts across three difficulty tiers (simple / medium / difficult). Describe how prompts were designed across tiers, and that the fixed ordering ensures apples-to-apples model comparisons.

4.3 Models & Configurations

List all models tested. Describe the parameter ranges (size, quantisation). Explain how you vary the maximum iteration count and how you determine when you've found the "optimal" cut-off — frame this as a quality-vs-cost tradeoff that you measure empirically.

Change: Expand this significantly. You now have a concrete model table to discuss (gemma4:e2b, qwen3:8b, qwen2.5-coder:14b, qwen3.5:9b) with an interesting emerging finding: reasoning capability matters more than parameter count. qwen2.5-coder:14b has more parameters than the others combined, yet it fails to fix issues because it lacks reasoning. This is now a key variable alongside model size. Add `temperature` and deterministically derived `seed` as explicit configuration parameters worth mentioning for reproducibility.

4.4 Validation Strategy

This is the most critical methodological section. Address the supervisor's concern directly: can the W3C validator actually detect semantic misuse of `<div>`? If it cannot (likely the case — W3C validates syntax, not semantics), you need to be transparent about this limitation and introduce a complementary evaluation. Options to discuss:

- A human evaluation rubric (e.g. annotators scoring semantic correctness)
- A secondary automated check (e.g. counting occurrences of semantic tags vs. `<div>` before and after)
- Possibly a separate LLM-as-judge evaluation

Clarify what "comparable output quality" means for RQ2 — define your quality metric explicitly.

Change: Add a subsection on the blind ablation control. This is a significant addition — for each run, the pipeline now runs both a `feedback` condition (model sees validator errors) and a `blind` condition (model is only told to "review and fix"). This lets you isolate how much improvement comes from the validator feedback itself versus simply giving the model a second attempt. This is important to call out as a deliberate design choice.

Also add: `check_validator_determinism.py` is used to verify the W3C validator itself is stable before trusting any before/after deltas, worth mentioning as a validity check.

4.5 Cost Metric

Define how you operationalise "cost" — API pricing per token, latency, or total tokens consumed per run. This addresses the supervisor's question about RQ2's cost dimension.

Change: Clarify that cost is proxied via generation latency and token counts (not API pricing), since all models run locally through Ollama. The results schema captures `gen_time_s`, `prompt_eval_count`, and `eval_count` per iteration. Note explicitly that no cloud/API-based model baseline is included in this pipeline (as stated in the repo's Scope section) — this is a scope limitation to acknowledge.

5 Experimental Setup (1 – 1.5 pages)

Describe the practical setup: hardware, software versions, Docker configuration, Ollama setup, which specific model versions/quantisations you used, and how you controlled for variability between runs. Include any pilot/exploratory results that informed your final setup (you already have some initial results with Gemma 4, Qwen 3, DeepSeek R1).

Change: Add mention of repeated trials per (model, prompt) combination, which enables variance estimation and confidence intervals rather than relying on single-run results. Also mention the `was_cleaned` flag, the pipeline strips markdown fences/preamble before validation, so formatting slip-ups are never counted as HTML errors (important for validity of the before/after comparison).

6 Results (3 – 4 pages)

6.1 RQ1 — Guardrail Effectiveness

Report error/warning counts before and after the guardrail, per model and per prompt. Show whether the guardrail improves semantic correctness — and be careful to distinguish between W3C validation errors and semantic correctness (flag this distinction explicitly here).

Change: Results now need to be broken down by condition (`feedback` vs `blind`) as well as by model and difficulty tier, not just before/after. The statistical analysis uses paired Wilcoxon tests and 95% confidence intervals, which should be stated. Also add error category breakdown as a dimension (missing-doctype, stray-tag, disallowed-attribute, etc.).

6.2 RQ2 — Cost-Efficiency

Compare smaller vs. larger models on quality metrics at various iteration counts. Show the quality-vs-cost tradeoff curve. Establish a baseline for large API-based models (or compare against a cited benchmark) as your supervisor suggested.

Change: Reframe this — the comparison is smaller vs. larger local models only, not local vs. cloud. Add the reasoning vs. non-reasoning dimension as a key finding here, since it turns out to be more predictive of success than parameter count alone. The scatter plots (model size vs. cost/quality, coloured by native thinking mode) directly address this.

6.3 Iteration Count Analysis

When does the loop converge? Is there a diminishing return after N iterations? This is where you answer the "optimal number of iterations" question.

Change: No major structural change, but the analysis now comes from `stats.py`'s iteration-cutoff analysis, which is worth naming explicitly as the source of the convergence findings.

7 Discussion (2 – 3 pages)

8 Conclusion (0.5 – 1 page)

References

- [1] Alexandra Souly, Javier Rando, Ed Chapman, Xander Davies, Burak Hasircioglu, Ezzeldin Shereen, Carlos Mougán, Vasilios Mavroudis, Erik Jones, Chris Hicks, Nicholas Carlini, Yarin Gal, and Robert Kirk. Poisoning attacks on llms require a near-constant number of poison samples, 2025. URL <https://arxiv.org/abs/2510.07192>.
- [2] Ravin Ravi, Dylan Bradshaw, Stefano Ruberto, Gunel Jahangirova, and Valerio Terragni. Llmloop: Improving llm-generated code and tests through automated iterative feedback loops. 07 2025. doi: 10.1109/ICSME64153.2025.00109.

Appendix

Full prompt dataset

Extended results tables

Flowchart (if not in main body)

Human evaluation rubric (if used)

Project timeline

- Full prompt dataset
- Extended results tables
- Flowchart (if not in main body)
- Human evaluation rubric (if used)
- Project timeline — your supervisor asked for this; an appendix is a natural place for it if it doesn't fit in the main body